Update **confidential** designator here

# Migration Toolkit for Virtualization

## Migrate VMs to OpenShift Virtualization – technical overview

Red Hat

Update **confidential** designator here

# Agenda

## Session outline

- Role of MTV between source and OpenShift Virt
- Operators, controllers, and custom resources
- Planning: providers, maps, migration plans
- Execution: CDI, disks, cutover concepts
- Prerequisites, limits, and documentation

Update **confidential** designator here

# Learning objectives

## Takeaways

- Explain MTV placement on the OpenShift cluster
- List core CRs and their order of use
- Describe how VM disks land on CSI PVCs
- Name key prerequisites for VMware or RHV sources



Red Hat

Technical audience

# What is MTV?

# Migration Toolkit for Virtualization

## Red Hat MTV (Forklift-based)

- Kubernetes-native migration controller on OCP
- Web console plus declarative CR workflow
- Sources: VMware vSphere, RHV, OpenStack (per matrix)
- Target: OpenShift Virtualization (KubeVirt) VMs

# Introducing the Vehicle: MTV

The automated, secure, and resilient engine designed specifically to bridge the gap between legacy source environments and your cloud-native target.

NotebookLM

# The 3-Stage Migration Lifecycle

### Stage 1: Discovery

Inventory clarity, scope sizing, and risk identification.

### Stage 2: Planning

Application assessment, dependency mapping, and infrastructure translation.

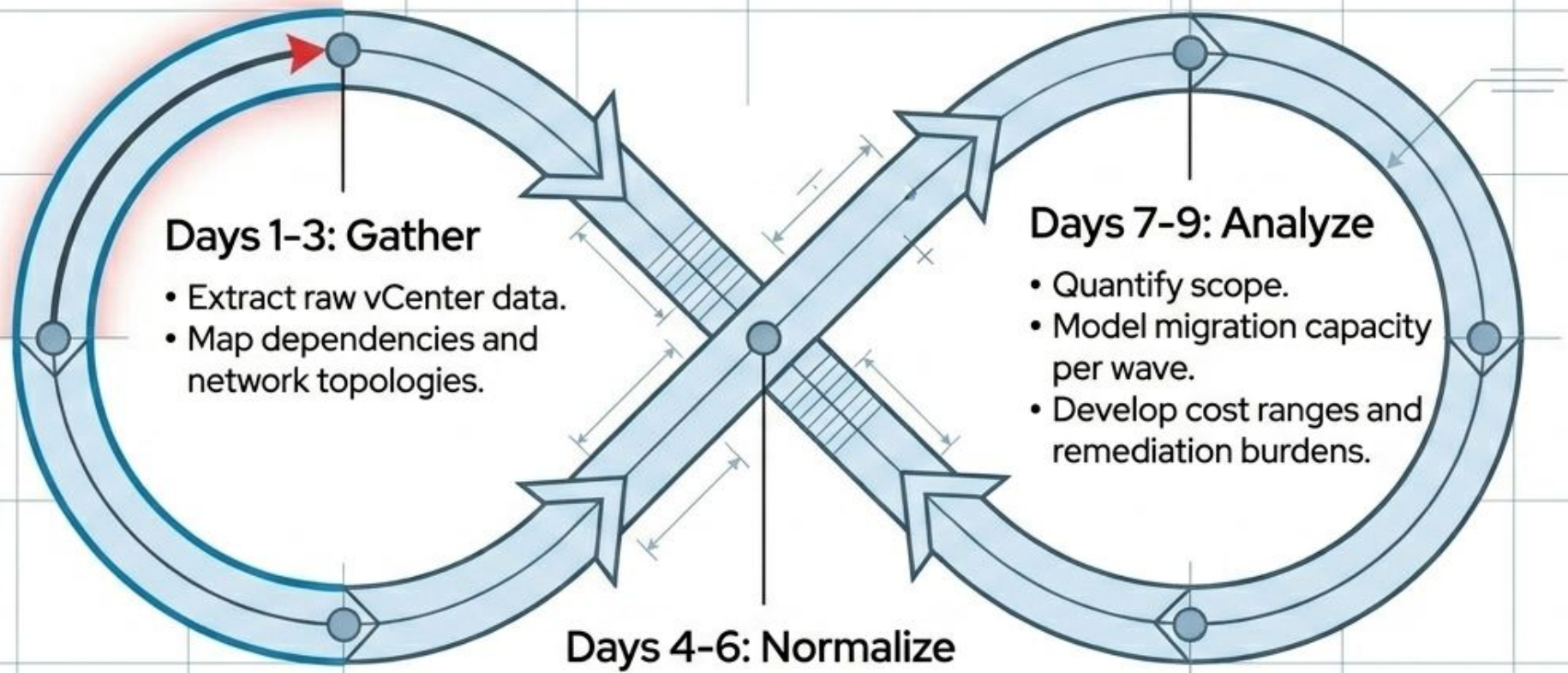
### Stage 3: Execution

Wave grouping, pilot migration, and bulk execution at scale.

NotebookLM

# Stage 1: The 2-Week Rapid Discovery

Delivering real outputs—not just a list of VMs, but a quantified framework for wave planning and risk mitigation.



### Days 1-3: Gather

- Extract raw vCenter data.
- Map dependencies and network topologies.

### Days 4-6: Normalize

- Standardize attributes.
- Categorize by migration complexity.
- Flag blockers (e.g., vSAN policy conversion).

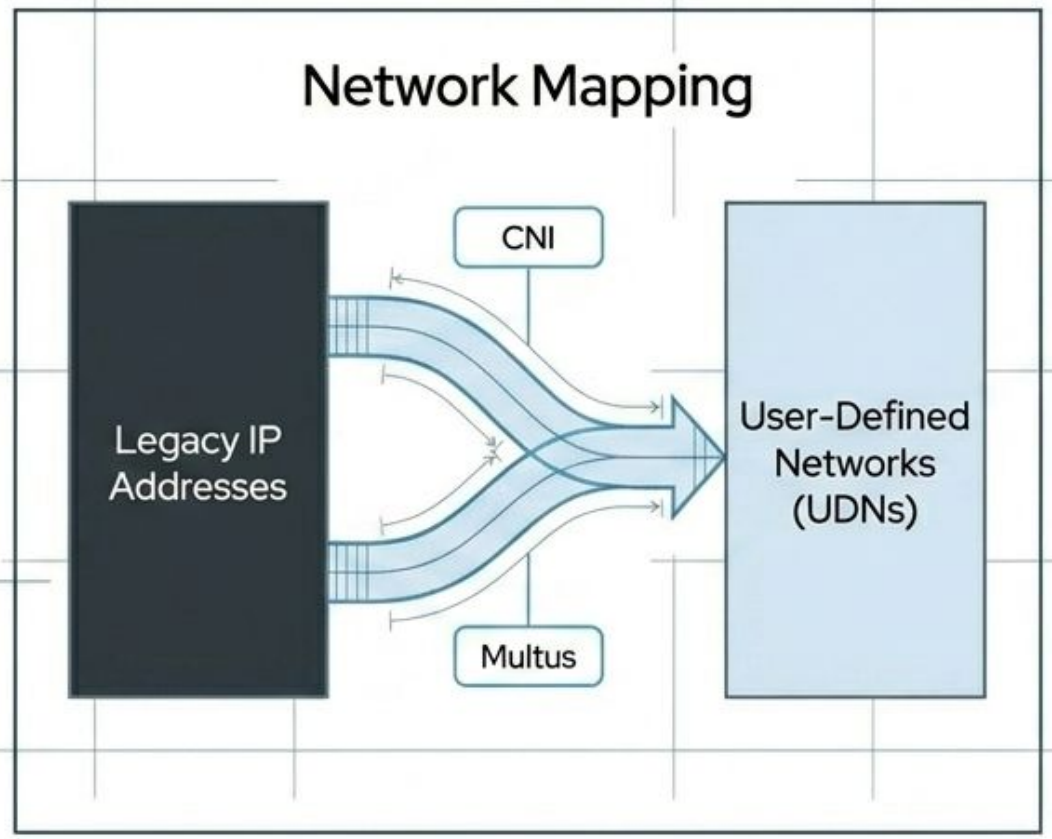
### Days 7-9: Analyze

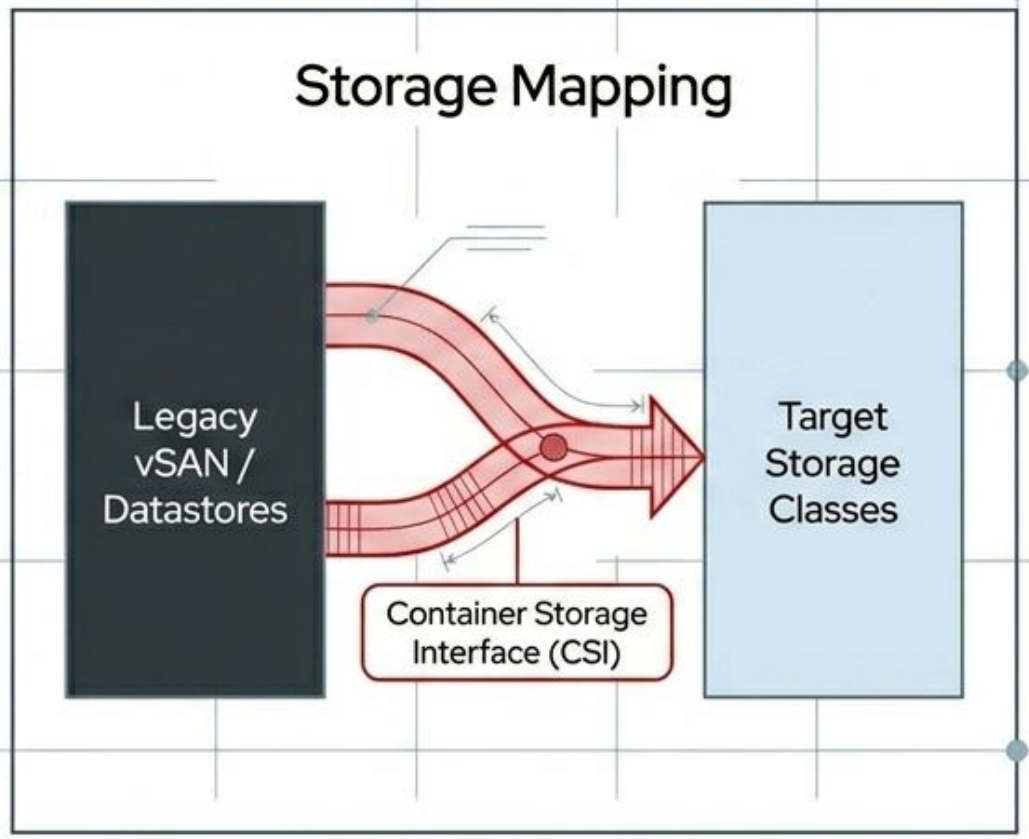
- Quantify scope.
- Model migration capacity per wave.
- Develop cost ranges and remediation burdens.

NotebookLM

# Stage 2: Storage & Network Mapping

**MTV natively translates and maps legacy configurations**, ensuring workloads retain their operational identity and dependencies in the new environment.




NotebookLM

# Why use MTV?

Update **confidential** designator here

# Operational benefits

## Compared to lift-and-shift scripts

- Reusable plans across hundreds of VMs
- Validated network and storage mappings
- Integration with OpenShift RBAC and projects
- Foundation for incremental or wave migrations



Red Hat

Still requires design and testing

# Architecture

Update **confidential** designator here

# End-to-end migration scope

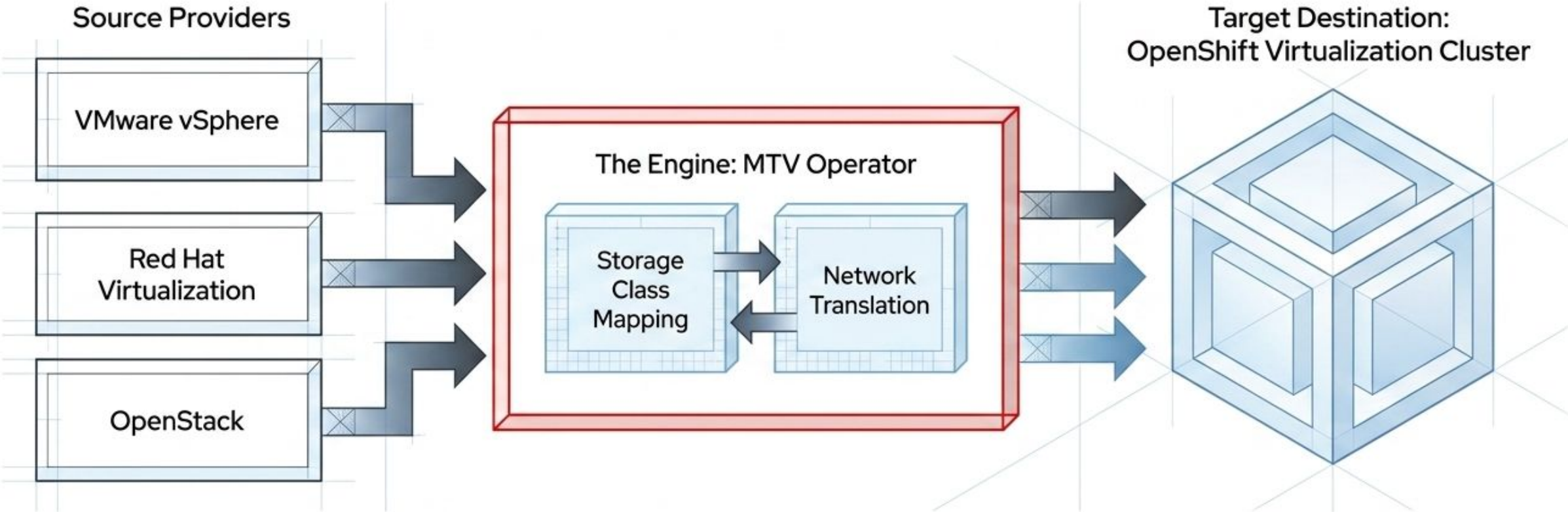
## Logical data flow

- MTV operators deploy into the OpenShift cluster
- Controller reaches source hypervisor APIs
- Disk data streams into namespace-scoped PVCs
- KubeVirt starts VMIs from those volumes



Firewall and DNS must allow controller traffic

# Architectural Flow of the Migration Engine

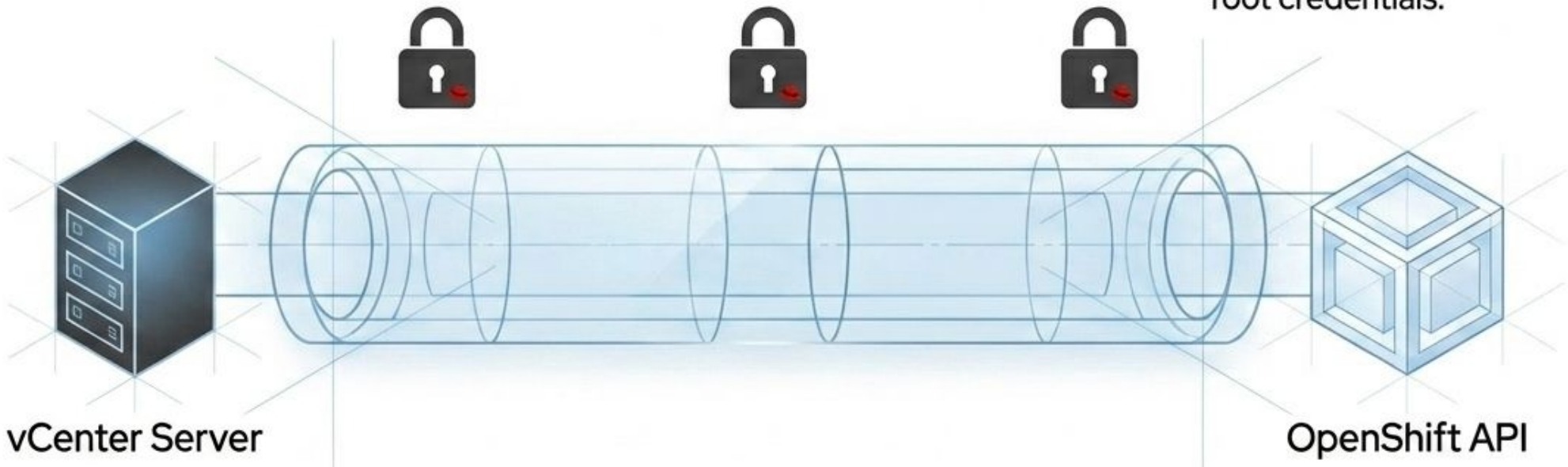



MTV connects directly to source provider APIs, analyzes the workload topology, translates legacy network/storage configurations to Kubernetes primitives, and orchestrates the live or cold transfer of disk data directly into OCP Persistent Volumes.

NotebookLM

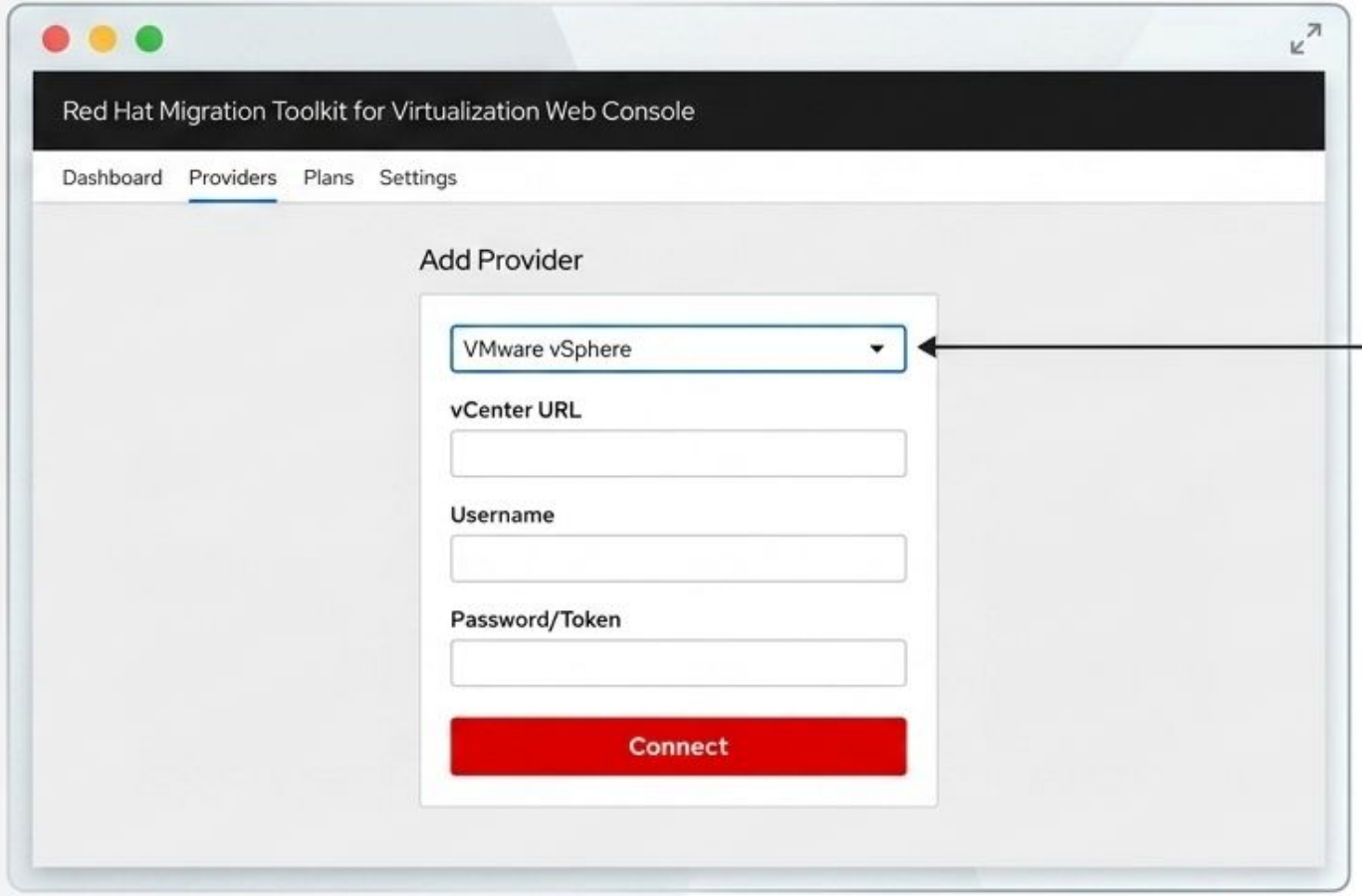
# Zero Trust Architecture & Provider Security

- **Identity-Based Access**
  Strict RBAC using the kubevirt.io:migrate role.
- **End-to-End Encryption**
  Secure, encrypted transport of all VM data blocks during transit.
- **Persistent Permissions**
  Utilization of long-lived service accounts and secure bearer tokens to safely interact with source APIs without exposing root credentials.




NotebookLM

# The MTV Interface: Connecting Providers




**Seamless API Integration.**

NotebookLM

# Execution path

# The 3-Stage Migration Lifecycle

### Stage 1: Discovery

Inventory clarity, scope sizing, and risk identification.

### Stage 2: Planning

Application assessment, dependency mapping, and infrastructure translation.

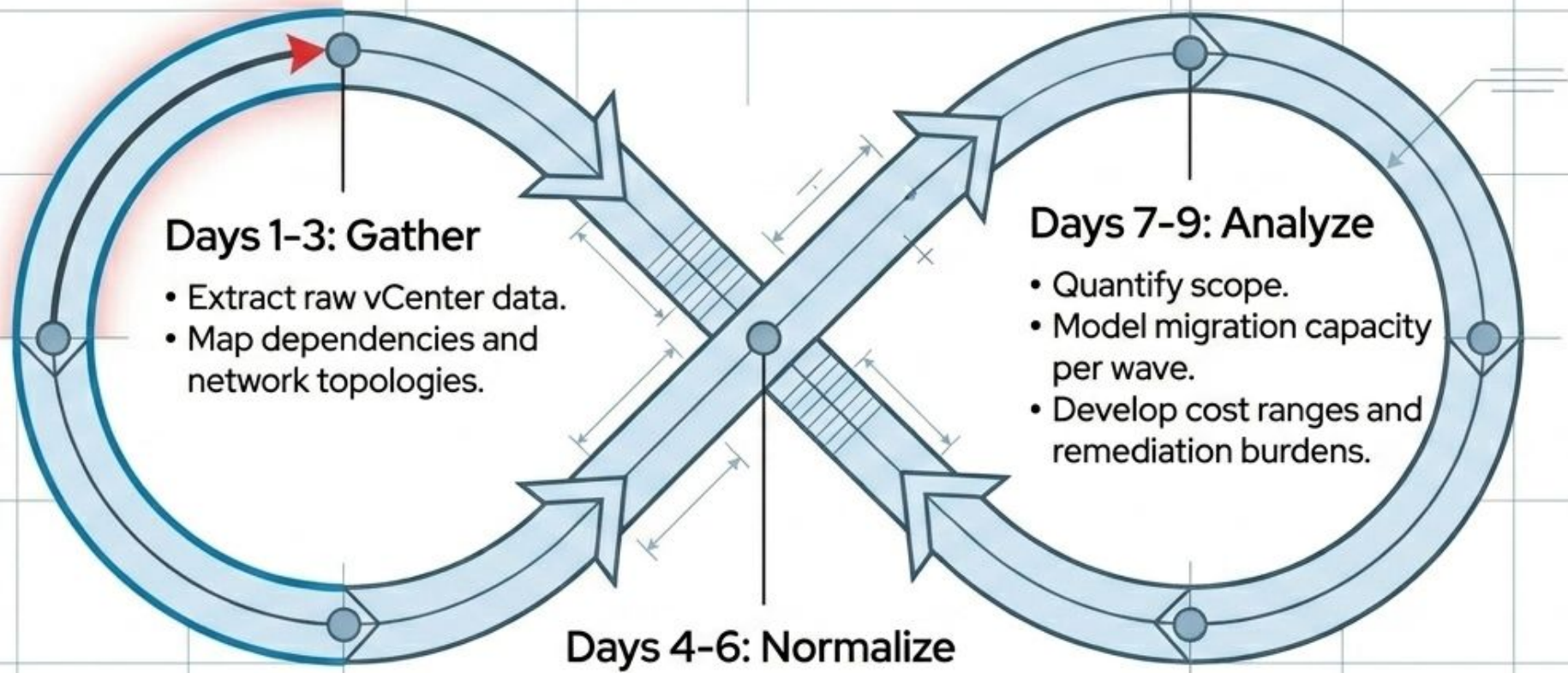
### Stage 3: Execution

Wave grouping, pilot migration, and bulk execution at scale.

NotebookLM

# Stage 1: The 2-Week Rapid Discovery

Delivering real outputs—not just a list of VMs, but a quantified framework for wave planning and risk mitigation.



### Days 1-3: Gather

- Extract raw vCenter data.
- Map dependencies and network topologies.

### Days 4-6: Normalize

- Standardize attributes.
- Categorize by migration complexity.
- Flag blockers (e.g., vSAN policy conversion).

### Days 7-9: Analyze

- Quantify scope.
- Model migration capacity per wave.
- Develop cost ranges and remediation burdens.

NotebookLM

# Stage 3: Bulk Execution & Wave Management

Migration Wave 1: Active

| VM Name | Progress | Status |
|---|---|---|
| web-server-01 | 50% | Copying disks... |
| db-prod-sql | 85% | Preparing target... |
| app-tier-03 | 30% | Copying disks... |
| analytics-node-02 | 95% | Finalizing sync... |

**Execution at Scale**

NotebookLM

Update **confidential** designator here

# Prerequisites checklist

## Before production waves

- OpenShift Virtualization + MTV operators healthy
- StorageClasses with free capacity and snapshots if used
- L3 connectivity: OCP nodes to vCenter and ESXi
- Time sync (NTP) across source and OpenShift





Review product-specific firewall port list

# Scope limits (honest view)

## What MTV is not

- Not a full application modernization framework
- Guest OS support follows virt-v2v / docs matrix
- Some hardware passthrough patterns unsupported
- Always test one representative VM first

# Thank you

MTV documentation |
https://docs.redhat.com/en/documentation/migration_toolkit_for_virtualization/latest/

OpenShift Virtualization |
https://docs.redhat.com/en/documentation/openshift_container_platform/latest/html/virtualization/

linkedin.com/company/red-hat

youtube.com/user/RedHatVideos

facebook.com/redhatinc

twitter.com/RedHat

Red Hat